24 單源最短路徑

Part VI Graph Algorithms

本章定位

最短路徑以relax的上界及前驅重建串聯算法；本章逐步證明Bellman–Ford、DAG與Dijkstra各自的適用條件，再用負環證書、差分限制及根樹驗證區分有限距離、不可達與無下界。

前置知識

Edge relaxation、負權與可達性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 維持 shortest-path 與 triangle-inequality 不變量。
- 分別分析 Bellman-Ford、DAG 與 Dijkstra。
- 把 difference constraints 化為負權圖。
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

距離有三種語義，負邊與負環不同

有限有向圖G=(V,E)，n≥1、s∈V，每條邊為有限實數權。路徑成本是邊權和；容許重複頂點的walk可以包含cycle。定義δ(s,v)為所有s-v walk成本的下確界：不可達為∞；可經過負cycle再到v則為−∞；其餘有有限最小值。源點的零長walk成本0，所以δ(s,s)≤0。

若到v的walk沒有可用負cycle，刪去正cycle會改善，刪去零cycle不變，因此存在至多n−1邊的簡單最短路。負邊本身不使問題無解；負cycle也只影響其**由s可達且能往後到達**的點，其他分支仍可能有有限距離。

單目的地可反轉全部邊後以目的地為source；單點對可取單源輸出的指定項。無向負邊若表示成兩條反向arc，便形成可往返的負closed walk，不能沿用「只是單一負邊」的直覺。

Relax：只改進暫定上界

```
INITIALIZE(s):
    對每個 v: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u,v)):
    若 d[u]!=∞ 且 d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u,e.ID)
        回傳 changed=true
    否則回傳 false
```

每個有限d[v]是實際s-v walk成本（對relax次數歸納），故d[v]≥δ(s,v)。只做嚴格改善，等值不改parent。平行邊必記ID；初始化Θ(n)，每次relax在精確算術單位成本模型下O(1)。有限整數實作須防溢位，∞要獨立標記；任意把負值截到某下限，會毀掉負環偵測。

一旦d達到有限δ，之後既不能增加，也不能低於δ，所以永遠固定。此處使用「有限」避免把−∞當成某次普通有限relax的輸出。

Bellman–Ford 的操作、早停與成本

```
BELLMAN-FORD(G,s):
    INITIALIZE(s)
    重複最多 n−1 輪:
        changed=false
        對固定順序每條 e∈E:
            若 RELAX(e)成功: changed=true
        若 changed==false: 回傳 d,parent,「無可達負環」
    對每條 (u,v)∈E:
        若 d[u]!=∞ 且 d[v]>d[u]+w(u,v):
            回傳「存在可達負環」
    回傳 d,parent,「無可達負環」
```

最後一輪是測試，不把某個仍在下降的有限d當最短距離交付。若一整輪沒有更新，所有可達邊已滿足d[v]≤d[u]+w；之後同樣輸入再掃也無變，故可早停。n=1仍要測自環，負自環會在最後檢查被發現。

固定全輪版本成本Θ(n+nm)，在m≥1、n≥2時常寫Θ(nm)；早停最好Θ(n+m)，最壞不變。額外d、parent為O(n)，輸入邊表O(n+m)。負環的完整分類與證書可另加下述DP版本。

完整追蹤：逆向邊序需要多輪傳播

頂點依(s,a,b,c,t,z)。邊按下列**固定順序**掃描：ct:3、at:8、ac:1、bt:12、bc:5、ba:−2、sa:4、sb:1；箭頭方向均由第一字母至第二字母。z孤立。

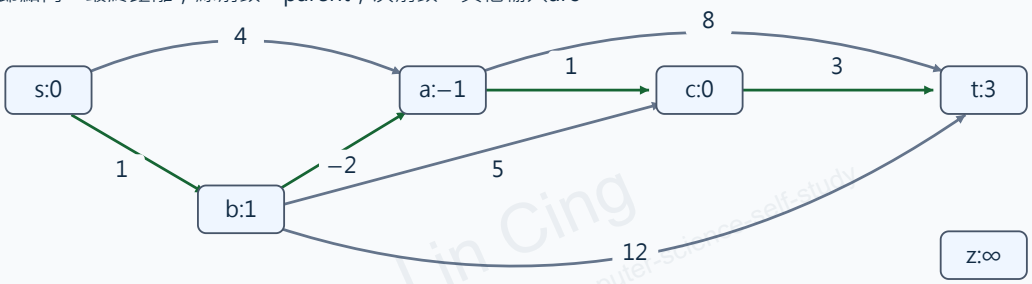
輪末	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

第2輪先以舊a=4產生t=12、c=5，稍後ba才把a改成−1。第3輪先ct得8，再at改7，ac才使c=0；第4輪ct才得3。第5輪無改動早停。最終parent為s→b→a→c→t，成本1−2+1+3=3。

完整追蹤：逆向邊序需要多輪傳播 （續）

Bellman–Ford：同一輸入與最終前驅

節點內 = 最終距離；綠箭頭 = parent；灰箭頭 = 其他輸入arc。



逆向掃描輪末：a 4→−1；c ∞→5→0；t ∞→12→7→3。

樹路 s→b→a→c→t：1−2+1+3=3；z 從 s 不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何 n−1 輪足夠，為何負環一定被看見

若無可達負環，任一可達v有簡單最短路$s=v_0,\cdots,v_k=v$，$k\le n-1$。第i輪必掃(v_{i-1},v_i)，且第i−1輪末$d[v_{i-1}]$已達δ；convergence給$d[v_i]=\delta$。即使輪內又處理更多邊，也不會破壞真值，所以到n−1輪全部正確。這只說原地版本能至少涵蓋i邊路徑，不說其值恰等於「最多i邊」的DP。

若有可達負cycle，各cycle頂點在n−1輪後都有限（任一可達點有≤n−1邊的可達簡單路，不要求該路最短）。若最後所有cycle邊都不能改善，沿圈相加$d[v]\le d[u]+w$，d項消去得到$0\le w(\text{cycle})<0$，矛盾。因此至少一條可達邊仍可改善。反之無負環時所有d已等δ，三角不等式排除任何改善。這是偵測的兩方向，不只以「值繼續降」作描述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

要交付負環證書：用明確的邊數 DP 重建

令$D_i[v]$為最多i邊的最小walk成本，$D_0[s]=0$、其餘∞。

$$D_i[v] = \min\left(D_{i-1}[v],\ \min_{(u,v)\in E}(D_{i-1}[u]+w(u,v))\right).$$

每一層只讀上一層；若保留舊值就記「skip」，若選一條邊就記其ID及上一層u。每個≤i邊walk不是≤i−1邊，就是刪去末邊後的一個≤i−1邊walk；反過來每個候選均合法，故兩向證明recurrence。

計算至第n層。若$D_n[v]<D_{n-1}[v]$，由記錄倒退n層得到**恰n邊**walk：少一邊便屬前一層，不會嚴格改善。分解該walk為一條簡單路及重複點截出的cycles。若所有cycles非負，刪掉它們得到≤n−1邊且成本不大的路，與嚴格改善矛盾。故至少一個截出的cycle權負，輸出其edge ID順序與總權就是可核對證書。

此完整記錄版時間$O(n+nm)$，$O(n^2+m)$空間；只存兩層可分類但不能保留上述回溯表。不要替此版本聲稱原地BF的O(n)輔助空間。

哪些點應標成 −∞

取seed集合$B=\{v:D_n[v]<D_{n-1}[v]\}$，沿**原圖方向**從B做BFS／DFS，所有到達點標−∞；其餘可達點取D_{n-1}，不可達保∞。

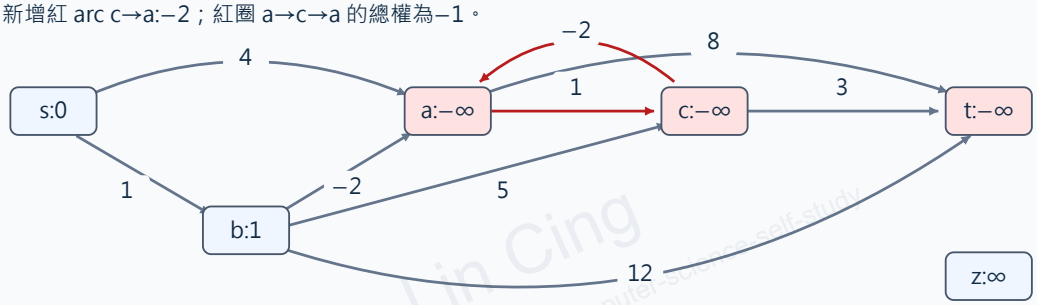
每個seed的n邊walk含負cycle，故seed及其後代都受影響。另一方面，任一可達負cycle的所有點在D_{n-1}已有限。若圈上沒有seed，則每邊$D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$，相加矛盾；故圈上至少一seed，所有受其影響的後代都會被遍歷。這證明標記範圍恰好，並非只標cycle本身。

沿前例加c→a:−2，a→c→a權1−2=−1，則(s,a,b,c,t,z)距離為(0,−∞,1,−∞,−∞,∞)。s、b到不了由cycle往回的出口，仍有限；z仍不可達。附帶一個獨立不可達負圈也不能讓s的BF誤報。

哪些點應標成 −∞ （續）

可達負環只影響其向前後代

新增紅 arc c→a:−2；紅圈 a→c→a 的總權為−1。



圈的後代 a,c,t 標−∞；b=1、s=0，因圈沒有回 s 或 b 的出口。

同一次負環偵測，不可把仍有限的分支或不可達 z 一起抹成−∞。

24.2 Single-source shortest paths in directed acyclic graphs

拓撲序讓每個候選都準時到達

```
DAG-SHORTEST(G,s):
    order=TOPOLOGICAL-SORT(G)
    若發現cycle: 拒絕此算法的輸入
    INITIALIZE(s)
    對 order 中每個 u:
        對 Adj[u] 每條 e: RELAX(e)
    回傳 d,parent
```

拓撲序保證任何s-v路上各邊依路徑先後被處理；path-relaxation使路徑終點取得δ。亦可對拓撲位置歸納：v被處理前，所有入邊起點均已固定且掃過，所以沒有未來才到的更好候選。不可達點保持∞。負邊可以有，因無cycle便無負cycle。拓撲排序及掃邊共Θ(n+m)，輔助O(n)。

前例本來就是DAG，取合法次序s,b,a,c,t,z：處理s後a4,b1；處理b後a−1,c6,t13；處理a後c0,t7；處理c後t3。一輪與BF四輪同結果，差在**頂點拓撲順序**而非改了成本定義。

關鍵路徑：最長路只在 DAG 中直接翻號

在DAG把每個權改−w，最短成本的負值便是原最長成本；或以L[s]=0、其他−∞並用max relax。一般有cycle圖不能這樣直接套DAG算法；正cycle會使最長walk無界。

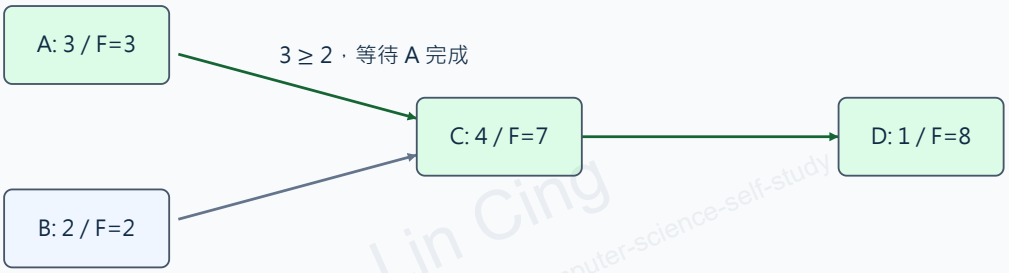
若工作是頂點，duration[v]≥0，依賴u→v表示u完成後v才能開始。最早完成F[v]=duration[v]+max({F[u]:u→v}∪{0})，依拓撲順序算；回溯實現最大值的前驅給critical path。

例如工作A:3、B:2、C:4、D:1，依賴A→C、B→C、C→D。F[A]=3、F[B]=2、F[C]=4+max(3,2)=7、F[D]=1+7=8；critical path A,C,D成本8。無限處理器且只受這些先後條件時，讓每工作一滿足前驅就開始，可達此下界8；有限處理器另有資源限制，不能只以此數當完整排程。

關鍵路徑：最長路只在 DAG 中直接翻號 （續）

工作 DAG：完成時間由較慢前驅決定

格內：工作 / duration / 最早完成 F；綠箭頭在 critical path。



F[C]=4+max(3,2)=7；F[D]=1+7=8。

無限處理器最早完成8；總工量10，一顆處理器仍至少需要10。

24.3 Dijkstra’s algorithm

取最小暫定距離後不再重新打開

此節明列所有邊w≥0。以indexed min-heap保存每個未取頂點各一項，key=d。初始化d[s]=0、其他∞、parent=NIL、inQ=true。

```
當 Q 非空:
    u=EXTRACT-MIN(Q); inQ[u]=false
    若 d[u]==∞: 停止（餘下皆不可達）
    對 Adj[u] 的 e=(u,v):
        若 inQ[v] 且 d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u,e.ID)
            DECREASE-KEY(Q,v,d[v])
```

已取S各點的d已是δ；與Prim相比，更新是**總路長d[u]+w**。已取點不更新也不重入Q。若要提早結束單點對查詢，須等目標真的extract，不能在它第一次被發現就停。

非負權究竟用在哪一行

對下一個有限extract u，若它為第一個錯誤，取一條真最短路s→⋯→x→y→⋯→u，y是路上第一個未取點，x已取。s第一個被取且d[s]=0；其餘情況x確已達δ並掃xy，故convergence保證d[y]=δ(s,y)。

從y到u的後綴權非負，得到δ(s,y)≤δ(s,u)。選最小d給d[u]≤d[y]；上界性給δ(s,u)≤d[u]。合併

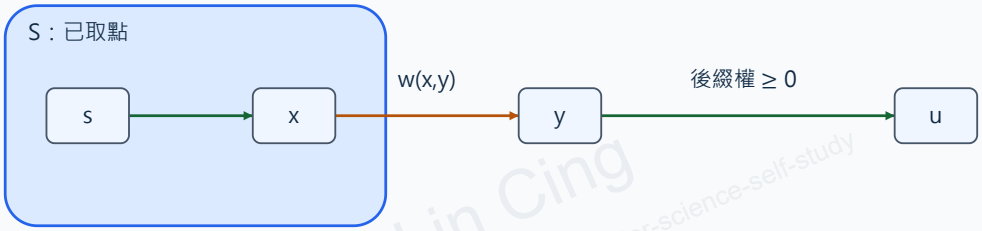
$$\delta(s,u) \le d[u] \le d[y] = \delta(s,y) \le \delta(s,u),$$

全部相等，矛盾，所以extract不會錯。零權邊與相同距離均可，無須嚴格遞增。∞被取時，若仍有可達未取點，該路第一未取點已有有限候選，與取到∞矛盾。

非負權究竟用在哪一行 （續）

Dijkstra：用第一個未取點夾住 u

只追蹤一條真最短路；橘箭頭是它第一次跨出 S 的 arc。



x 已固定且掃過 xy ⇒ d[y]=δ(s,y)。

最小提取 d[u]≤d[y]，非負後綴 δ(s,y)≤δ(s,u)，上下界夾成相等。

數值追蹤與負邊的真正反例

將BF前例只有ba改為+2。逐輪掃完：

extract	未取的有限 d(parent)	已固定距離
s0	a4(s),b1(s)	s0
b1	a3(b),c6(b),t13(b)	s0,b1
a3	c4(a),t11(a)	再加a3
c4	t7(c)	再加c4
t7	無；z∞	再加t7

最短樹s→b→a→c→t，路長1+2+1+3=7。候選t曾13→11→7，第一次發現t不能當最短值。

負邊反例用s→a:2、s→b:5、b→a:−4、a→t:2。取s後先固定a2並令t4，再固定t4，最後b5才提供a1與真正t3。標準不重開版本維持錯值；即使容許更新已取a而不重掃其出邊，t仍錯。此圖為DAG、沒有負環，失敗恰在δ(s,b)=5>δ(s,a)=1，使證明的非負後綴比較失效。

Queue、輸入表示與精確算術成本

indexed binary heap：建heap O(n)、n次extract共O(n log n)、每邊最多一次decrease共O(m log n)，總O((n+m)log n)，n=1特判。Fibonacci heap將decrease降至O(1)攤銷，總O(m+n log n)。每個heap項須能以handle定位，inQ用O(1)陣列查。

線性掃全部未取d配adjacency list為$O(n^2+m)$，simple graph可簡化$O(n^2)$。若以lazy heap推入所有新候選，項目可達m，需另證O((n+m)log(m+1))及O(n+m)空間，不能套只有n項的indexed版本成本。例程使用線性掃key及adjacency list做數值驗算。

上述是實數RAM模型；B位元整數、無負環的簡單路成本需O(B+log n)位元，精確加法／比較成本應另外計入。浮點誤差與任意epsilon可能改變等值和負環判斷；數學正確性並非自動等同浮點穩定性。

24.4 Difference constraints and shortest paths

限制的方向來自移項，不憑語感猜

限制$x_j - x_i \le b$對應arc $i \to j$:b，因為它等同$x_j \le x_i + b$。每列係數矩陣只有一個+1與一個−1，變數可為任意實數。若寫$x_j - x_i \ge b$，先轉成$x_i - x_j \le -b$。等式拆成兩個反向不等式。

加super-source q→每個變數點，權0；q無入邊，確保所有原負cycle都可達。無負cycle時設$x_i = \delta(q,i)$，由每邊三角不等式得$x_j - x_i \le b$，所有限制成立。若存在負cycle，把對應限制相加，左邊每個變數正負各一次全消，得到$0 \le \Sigma b < 0$，矛盾。這建立可行⇔無負cycle。

原n變數m限制，加q後n+1點、m+n邊，BF為$O(n^2+nm)$。改為原圖所有d初設0，相當已完成q的零邊，跑至多n−1輪原邊及一輪檢查，成本O(n+nm)，包括m=0時仍需初始化，不寫成0。

完整可行與不可行數例

限制為$x_2-x_1\le -2$、$x_3-x_2\le -1$、$x_1-x_3\le 4$，邊為1→2:−2、2→3:−1、3→1:4。全零開始依此序：第一次掃得到(0,−2,−3)，因−3+4=1≥0不改x_1；第二輪不變。

逐條驗證：−2−0=−2≤−2，−3−(−2)=−1≤−1，0−(−3)=3≤4。全體加3得(3,1,0)仍可行，因差值不變。一般可行域沿全一向量平移，沒有固定原點。

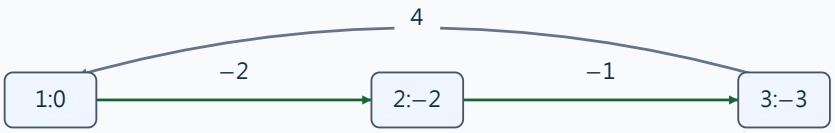
若把最後限制改$x_1-x_3\le 2$，三式相加為$0\le -2-1+2=-1$，無解。負cycle為1→2→3→1；不能只報某個候選不符合，就推所有候選都不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整可行與不可行數例 (續)

同一組限制：可行向量與負環矛盾

每條 arc i→j:b 對應 xj−xi≤b；圖內值為可行 x。



逐條差值：−2−0=−2；−3−(−2)=−1；0−(−3)=3≤4。

若最後權4改成2：三式相加得 0≤−2−1+2=−1，矛盾。

原圈總權1可行；改後總權−1無解。整體平移不改任何差值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些延伸仍保留這個證明

等式$x_j-x_i=b$拆成$i\to j:b$與$j\to i:-b$，兩向合起恰強制相等。欲表示$x_i\le c$及$-x_i\le c$，增加一個reference點r，用$r\to i:c$及$i\to r:c$；求解後整體平移使$x_r=0$，才能把它當絕對界，不能獨立令全部d任意平移卻忘記此基準。

整數右端時BF從整數0用加法/min計算，所得可行解自動整數。若要求**所有**變數整數且右端為實數，先將每個b換成floor(b)：因x_j-x_i為整數，兩個限制等價；再檢查負cycle。若只有部分變數要求整數，這個逐列floor方法不再自動適用。

所有$d\le 0$的規範下，最短距離還是逐座標最大的可行向量：任意可行x沿$q\to\cdots\to v$路反覆用限制，$x_v\le$路徑成本，故$x_v\le\delta(q,v)$。這是加入$x_i\le 0$後的結論，原平移不受限時Σx_i可任意增加。

24.5 Proofs of shortest-paths properties

最優子結構、三角不等式與收斂的完整鏈

假設本段沒有從s可達的負cycle。若一條有限最短路的子段不是最短，用更便宜子段替換便產生更便宜s-v walk，矛盾。對任何arc $u\to v$，接一條s-u最短路再接該arc是候選，故$\delta(s,v)\le\delta(s,u)+w$；u不可達時右邊$\infty$不增加限制。

上界性由初始化及每次有限d為walk成本證明；不可達點若曾變有限，就有實際walk，矛盾，故永保∞。Relax當下結束必有$d[v]\le d[u]+w$；**只保證此當下**，u日後再降，該邊可能要重掃。

若u已真最短且$u\to v$接在某條最短路上，該次relax後$d[v]\le\delta(s,u)+w=\delta(s,v)$，又上界性給反向，故v固定於真值（convergence）。依最短路的arc順序套此論證，第一點s已真值，歸納每次把下一點固定；插入其他relax亦只降上界且不破真值，故path-relaxation成立。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減

假設某次嚴格relax u→v第一次把parent子圖變成cycle。此前已有parent路v→⋯→u。每條舊parent arc x→y，在最後設定parent[y]時有d[y]=d[x]+w；其後y未更新，x只可能下降，所以本次更新前 d[y]≥d[x]+w。沿舊路相加得d[u]≥d[v]+w(v→⋯→u)。

新更新又要求d[v]>d[u]+w(u,v)，合併得cycle總權<0。路上各點皆有限，故由s可達，與假設矛盾。負自環亦直接由嚴格改善推出權負。所以parent子圖始終無cycle。

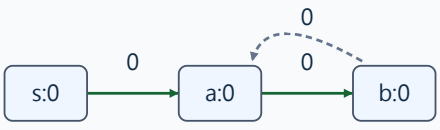
任何有限v≠s必由relax得到非NIL parent；s因d[s]=δ(s,s)=0固定，parent[s]=NIL。沿parent回溯，有限點的parent也有限；有限圖且無cycle，必停在唯一沒有parent的s。每個點只有一個parent，所以得到以s為根、包含所有已發現有限點的樹。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減 （續）

零權邊：距離相等仍須檢查 parent 根結構

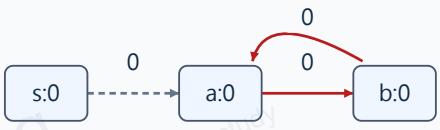
原圖 s→a:0、a→b:0、b→a:0；兩側全部距離都是0。

合法 parent：由 s 有根路



parent[a]=s，parent[b]=a

錯誤 parent：a,b 互為父



parent[a]=b，parent[b]=a

綠 = 合法樹 parent；紅 = 錯誤cycle；灰虛線 = 不在所選parent中的原arc。

Tight等式只驗成本，仍要從 s 遍歷 parent children，確認根連通且無環。

正確距離加合法 parent 才能交付最短路樹

當全部有限d已等δ，對當前parent arc u→v，最後更新時間給d[v]≥d[u]+w；三角不等式給δ(s,v)≤δ(s,u)+w，代入d後兩向合成等號。沿s-v樹路相加，內部d項望遠鏡消去，路權=d[v]−d[s]=δ(s,v)。不可達點仍∞且parent=NIL，故三項最短路樹條件全部成立。

不能改說沿parent距離嚴格下降：零權arc可以相等，負權arc沿正向還會下降；無cycle依賴剛才的嚴格更新時間證明。若不是由這種relax歷史產生，而是事後對每點任選tight predecessor，零權圈可被選成parent cycle。例如s→a:0、a→b:0、b→a:0，d均0；設parent[a]=b、parent[b]=a每邊都tight，卻沒有根路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性時間驗證輸出，連最優性也有證書

給候選d與parent edge IDs，先在原圖從s算可達集合R；要求R內d有限、d[s]=0，R外d=∞且parent=NIL。驗每個v∈R−{s}的parent確為輸入arc u→v，且d[v]=d[u]+w；用parent children做一次DFS，確認從s恰好遍歷R而無重複，parent[s]=NIL。再掃所有u∈R的輸入arc，驗d[v]≤d[u]+w。

樹路的tight性給一條成本d[v]的真路，故δ(s,v)≤d[v]。另一方面沿任意s-v walk把所有邊不等式相加得d[v]≤該walk成本，故d[v]≤δ(s,v)。兩向相等；可達負cycle也會違反相加不等式，所以證書拒絕它。驗證時間Θ(n+m)、空間O(n)，不需重跑BF或Dijkstra。

這同時解釋為何只驗邊不等式不夠（數值可能太小、沒有實現路徑），只驗tight parent不夠（可能零圈或另有更便宜輸入arc）；兩種證據各負責一個方向。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 24。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study